

Person 1 -- Core AI Pipeline and Data

NHAA Trauma Triage System -- Standalone Build Specification

How to Use This Document

This is a complete, standalone build specification. Read every phase in order and build it directly. Do not wait for, or require live coordination with, any other team member -- wherever this document needs something another person is building, it tells you exactly what mock/stub version to build for yourself so your work is fully testable on its own. All five team members are working from a shared JSON message contract (reproduced below); as long as everyone's real output matches this contract, all five pieces will merge correctly at the end without needing to have talked to each other during the build.

Your Role

You own the brain of the system: turning audio into a transcript, turning that transcript into eight risk scores, turning those scores into one SVI number.

The Shared Contract (Do Not Deviate From This Shape)

Every message you emit must follow this shape exactly:

```
{
  "type": "transcript_update",
  "text": "string, the transcript so far",
  "words": [{"word": "string", "start": 0.0, "end": 0.0}],
  "lang_detected": "hi | en | hinglish",
  "risk_vector": {
    "scores": {"acute_distress": 0.0, "depression": 0.0,
"self_harm_risk": 0.0,
    "fear_of_retaliation": 0.0, "intimidation": 0.0,
"dissociation": 0.0,
    "social_isolation": 0.0, "chronic_trauma_indicators":
0.0},
    "confidence": {"acute_distress": 0.0},
    "matched_evidence": {"acute_distress": ["exact phrase or silence
event that fired"]}
  },
  "svi": {"value": 0.0, "bucket": "low | moderate | high |
critical"},
  "silence_events": [{"start_sec": 0.0, "duration_sec": 0.0,
"placement": "mid_sentence | inter_word | inter_turn"}]
```

}

Other components (built independently by other people) will consume this exact shape. You will never see their code during the build. Build a small script that prints sample messages in this exact format so you can visually confirm correctness yourself.

Phase 1 -- Fetch Every Real Dataset (Do This First, Exact URLs Below)

Fetch and process these five real sources. Do not substitute, do not skip, do not invent placeholder data -- these are real, verified, currently-accessible sources.

1. **PHQ-9** (for the depression dimension). Go to **www.phqscreeners.com**. Download the PHQ-9 in English and in Hindi (the site hosts translated versions -- select Hindi from the language list). Extract all 9 item texts from both language versions.
2. **GAD-7** (for the acute_distress dimension). Same site, **www.phqscreeners.com**. Download GAD-7 in English and Hindi. Extract all 7 item texts.
3. **PC-PTSD-5** (for the chronic_trauma_indicators dimension). Go to **www.ptsd.va.gov/professional/assessment/screens/pc-ptsd.asp**. Download the instrument text. This one is English-only at the source; you will need to translate the extracted phrases into Hindi/Hinglish yourself in Phase 1 step 6 below.
4. **ICON 2021 Hindi-English Suicidal Ideation Corpus** (for the self_harm_risk dimension). Go to **aclanthology.org/2021.icon-main.14/**. Download the PDF. Check the paper's final page and footnotes for a linked data repository (GitHub/Zenodo); if a repository link exists, download the labeled dataset directly. If no public repository link is found, extract example phrases quoted directly in the paper's results/examples sections instead.
5. **DAIC-WOZ** (acoustic reference, for the acute_distress and general acoustic-feature validation). Go to **dcapswoz.ict.usc.edu**. Submit the access-request form using an academic email address. This will not grant access today -- proceed with the rest of your build without waiting for approval; treat this as a background task running in parallel.
6. **Dissociative Experiences Scale (DES)** (for the dissociation dimension, in addition to the silence-based signal built in Phase 2). No downloadable dataset exists, but the DES is a well-known, widely published psychology instrument. Search "Dissociative Experiences Scale items" and read two or three academic overview articles describing its symptom categories (memory gaps, feeling detached from one's own body, not recognizing familiar surroundings, losing track of time during conversation). Do not copy any copyrighted item wording verbatim -- write your own natural-language phrase variants inspired by these documented

symptom categories, in Hindi, English, and Hinglish (e.g., "mujhe kuch yaad nahi" / "I don't remember what happened" / "everything feels unreal").

7. **UCLA Loneliness Scale** (for the `social_isolation` dimension). Same approach: search "UCLA Loneliness Scale items" and read published descriptions of its concept categories (feeling left out, lacking companionship, feeling isolated from others). Write your own phrase variants in Hindi, English, and Hinglish inspired by these categories (e.g., "koi baat nahi karta" / "no one talks to us anymore" / "samaj ne bahiskar kar diya").
8. **fear_of_retaliation and intimidation** have no existing dataset or named instrument anywhere. Build a `keywords/fear_of_retaliation.json` and `keywords/intimidation.json` file yourself using this exact process: write 10 short example phrases per dimension per language (Hindi, English, Hinglish) describing what someone afraid of retaliation, or someone being actively intimidated, might plausibly say on a call. Ground every phrase in general, publicly known patterns of caste-based intimidation (do not invent identifying details of any real case) -- for example: "wo phir aayenge" (they will come back), "mujhe mana kiya tha call karne se" (I was told not to call), "agar main bolungi to mera parivaar khatre mein hoga" (if I speak, my family will be in danger). Write these yourself directly; do not wait for anyone else to provide them.

Deliverable for Phase 1: a `keywords/` folder containing exactly eight JSON files -- `acute_distress.json`, `depression.json`, `self_harm_risk.json`, `fear_of_retaliation.json`, `intimidation.json`, `dissociation.json`, `social_isolation.json`, `chronic_trauma_indicators.json` -- one per dimension, no exceptions, each entry formatted as `{"phrase": "...", "language": "hi|en|hinglish", "source": "PHQ-9 item 3" or "self-authored, caste-intimidation pattern" or "DES concept category" or "UCLA Loneliness Scale concept category"}`.

Phase 2 -- Build the Four Core Modules, Tested Alone

Build each of these as an independent, independently-testable Python module. Write a small test script for each one before moving to the next.

asr.py -- Use OpenAI's Whisper ("small" model, via `pip install openai-whisper`) for English and Hinglish with `word_timestamps=True`. Use `ai4bharat/wav2vec2-large-xlsr-53-hindi` (via `pip install transformers torch`, loaded through the `transformers` pipeline) for Hindi-only input. Test: run it on a 5-second silent audio array (all zeros) and confirm it returns `transcript_available: false` cleanly rather than crashing.

silence_detector.py -- Compute short-time RMS energy over the audio buffer in 20ms frames. Any run of frames below a threshold (start at 0.010, tune later) lasting 0.5 seconds or more is a silence event. Classify placement by comparing the silence's start/end time against the word timestamps from `asr.py`: if it falls between two words with no clause-ending punctuation before it and lasts over 1.2 seconds, classify as `mid_sentence`; otherwise `inter_word` or `inter_turn`. Test: build

a synthetic array (1s random noise at amplitude 0.08, 2s near-silence at amplitude 0.002, 1s random noise at 0.08) and confirm the 2-second gap is detected and classified correctly.

risk_scorer.py -- Load all eight of your Phase 1 keyword JSON files -- confirm all eight load correctly with a startup check that raises an error if any of the eight expected files is missing. For each dimension, scan the transcript text for matches (case-insensitive substring match is sufficient for a first version). Score = a value between 0 and 1 that increases with the number of distinct matched phrases (e.g., $\min(1.0, 0.5 * \text{number_of_matches})$). Every non-zero score must include the literal matched phrase(s) in matched_evidence. Fold in silence_detector.py's output: if a mid_sentence silence over 2 seconds is detected, raise the dissociation score by at least 0.5 in addition to any lexical score. Test with a full sample transcript containing at least one real phrase per dimension and confirm every dimension fires correctly with the right evidence attached.

Also compute a per-dimension confidence value in this same module: confidence = 0.9 if 2 or more distinct phrases matched, 0.6 if exactly 1 phrase matched, 0.3 if only the silence signal contributed with no lexical match at all. Also compute one overall system-wide confidence value as the average of all eight per-dimension confidences, and include it in your output as "overall_confidence": 0.0. This is a distinct field from the per-dimension confidence dict already in the shared contract -- add it as a new top-level field in your output JSON, and note this addition in your Phase 5 README since it's an addition to the originally circulated contract shape that Person 2 and Person 3 will need to know about.

svi_aggregator.py -- Implement this exact weighted formula:

```
SVI = 0.30*self_harm_risk + 0.22*fear_of_retaliation +
0.18*acute_distress
    + 0.14*dissociation + 0.10*chronic_trauma_indicators +
0.06*social_isolation
    (add depression and intimidation with 0.0 base weight for now if
the formula needs
    to sum exactly to 1 across all eight; adjust proportionally and
document your final
    weights clearly in code comments)
```

Bucket mapping: 0-25 Low, 26-50 Moderate, 51-75 High, 76-100 Critical. Add an exponential moving average ($\text{smoothed} = 0.55 * \text{new_raw_score} + 0.45 * \text{previous_smoothed_score}$) so the SVI reflects the call so far, not just the latest instant. Test: feed it three successive score sets simulating an escalating call and confirm the SVI climbs realistically rather than jumping erratically.

Phase 3 -- Wire the Four Modules Into One Pipeline

Build a single Python script that: accepts an audio file path as input, chunks it into 2-second windows, runs each window through asr.py then silence_detector.py then risk_scorer.py then svi_aggregator.py in sequence, and prints the final JSON message (matching the shared contract exactly) after each chunk. Add a timestamp measurement (time.perf_counter()) around each of the four stages and include

"latencies_ms": {"asr": 0.0, "silence": 0.0, "risk_scorer": 0.0, "svi": 0.0} in the output message. Test this full pipeline against a real recorded voice clip of someone reading one of your Phase 1 vignette-style sentences aloud.

Phase 4 -- Real-World Hardening

Test your Phase 3 pipeline against: a fully silent audio file (no speech at all -- confirm it does not crash), a clip with heavy background noise, and a clip where someone deliberately switches between Hindi and English mid-sentence. For each case, confirm the output still matches the shared contract shape exactly, with `transcript_available: false` where appropriate rather than an error being thrown. Fix any crash you find. This phase has no dataset to fetch -- it is entirely about making your own Phase 2/3 code robust against messy real input, and it will likely take longer than it looks.

Phase 5 -- Package for Merge

Wrap your full pipeline as a single importable Python module (`nhaa_core_pipeline.py`) with one clear entry-point function: `process_audio_chunk(audio_array, call_id, language) -> dict` returning exactly the shared-contract JSON shape. Write a short `README_core_pipeline.md` describing exactly how to call this function and what it returns, so that whoever is merging all five people's work at the end can wire your module in without needing to read your internal code. Include your real, measured latency numbers (from Phase 4 testing) in this README.